

PROTEGE SHORT COURSE

From Text to Ontology

How AI drafts a grounded ontology that you can open and edit in Protégé.

Josef Hardi

June 2026



Why should you care?

Almost all of the world's knowledge lives in **unstructured text** (papers, reports, recipes). An ontology represents that knowledge as classes, relationships, and axioms that a machine can reason over. But translating the text into that structure is still **done by hand**.

1

Manual & slow

Every class, synonym, and link is typed in by hand, one term at a time.

2

Expert bottleneck

Authoring an ontology needs domain knowledge, formal modelling skill, and vocabulary expertise. Finding that person is rare, costly, and serial.

3

Grounding is tedious

Matching recognized names to reference ontologies by hand, thousands of times over is exhausting.

*What if an LLM produced the first draft that **already grounded in reference ontologies** in minutes, leaving you to review instead of transcribe?*

Meet OntoGPT & SPIRES

OntoGPT

An open-source framework (Monarch Initiative) that extracts structured, ontology-grounded knowledge from text using LLMs.

SPIRES

Structured **P**rompt **I**nterrogation & **R**ecursive **E**xtraction of **S**emantics — OntoGPT's core extraction method.

1

Schema-first

You declare the target schema in LinkML. The LLM fills it so it never invents the structure.

2

Recursive

Nested objects are extracted level by level, each with its own focused prompt.

3

Grounded

Entity names are mapped to real reference ontology IRIs (FoodOn, UO, UBERON, MONDO).

You describe the schema; the LLM fills it

```
classes:
  Recipe:
    label
    ingredients: { range: Ingredient }
    steps:       { range: Step }
  Ingredient:
    food_item: { range: FoodItem }
    amount:    { range: Quantity }
  FoodItem:
    food: { range: FoodType }
  Quantity:
    unit: { range: Unit }
  FoodType:
    id_prefixes: [ FOODON ]
    annotators:  sqlite:obo:foodon
  Unit:
    id_prefixes: [ UO ]
    annotators:  sqlite:obo:uo
```

A [LinkML](#) **template** names the classes you want and which ontology grounds each field. Changing the schema will change what's extracted. No prompt-engineering required.

1

Classes = your target types

Recipe, Ingredient, FoodItem, Quantity.

2

Nested ranges = recursion

A slot whose range is another class triggers a deeper extraction.

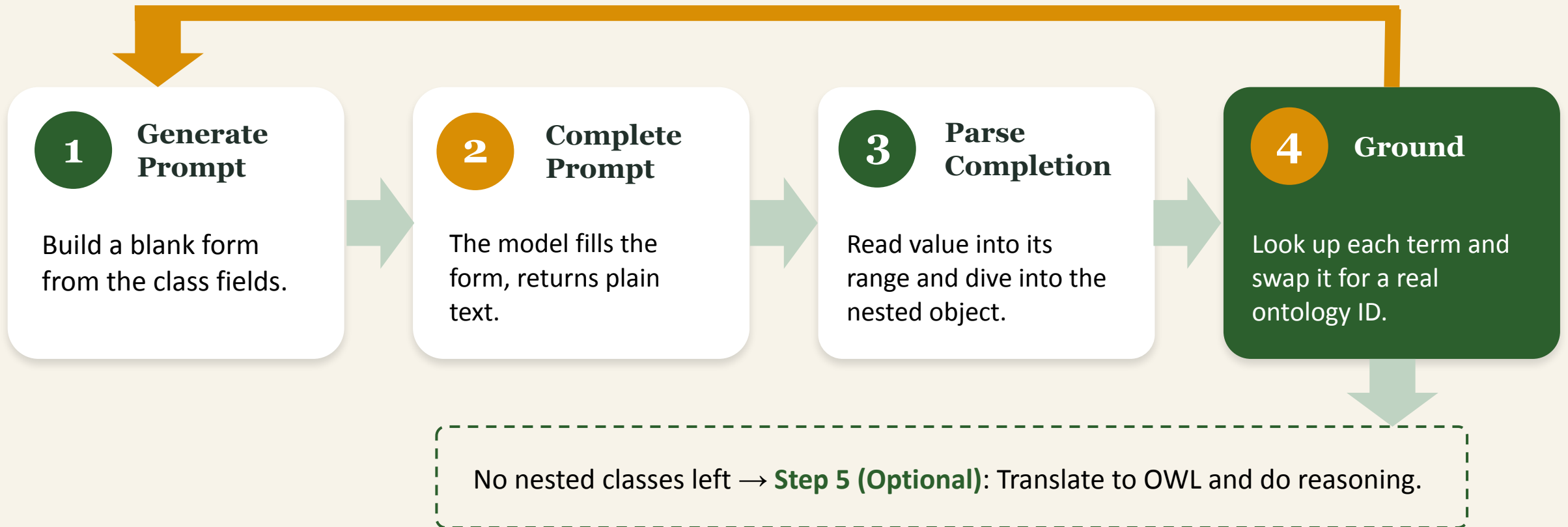
3

Annotators = grounding source

Tells SPIRES which ontology to map each term to.

Structured prompts, applied recursively

RECURSE – Repeat step 1-4 for each nested class



Process the Ingredient through the Four Steps

Input text: “2 pounds of ground turkey”

1

Generate Prompt

Build a blank form from the class fields.

Split this text into the following fields:

```
food_item: <the food item>
amount: <the quantity, e.g., 2 lbs>
```

Text: 2 pounds ground turkey

2

Complete Prompt

The model fills the form, returns plain text.

```
food_item: ground turkey
amount: 2 pounds
```

3

Parse Completion

Read value into its range and dive into the nested object.

```
food_item: { range: FootItem }
(after recursion)
    food: turkey    ·    state: ground

amount: { range: Quantity }
(after recursion)
    value: 2    ·    unit: pound
```

4

Ground

Look up each term and swap it for a real ontology ID.

```
turkey → FOODON:02020418
pounds → UO:0010034
```

Let's Watch in Action!

1

Food Recipe

Using predefined Recipe schema from OntoGPT to build an ontology about **Spinach and Feta Turkey Burger** recipe.

2

Using Local Model

Using a local model (**Gemma 4**) instead of cloud-based LLM model.

3

Grocery Ontology

Using a custom schema to define **Fudge Mint Cookie** from our short course exercise.

DEMO MATERIALS

<https://github.com/protegeteam/ai-for-ontology-engineering>

Run the OntoGPT `extract` Command (GPT-4o)

```
$ ontogpt extract \  
-i allrecipes-spinach-feta-turkey-burgers.txt \  
-t recipe.Recipe \  
-O yaml \  
-o spinach-feta-turkey-burgers.yaml
```

PARAMETERS

- i Path to the input text file to extract from.
- t The schema/template to extract against (required). Example: `recipe` is the predefined template and `.Recipe` selects the root class within it.
- O Output serialization format. One of: `json`, `yaml`, `pickle`, `md`, `html`, `owl`, `turtle`, `jsonl`, `kgx`, `csv`, `tsv`.
- o Where to write the result. Without `-o`, output goes to stdout (the terminal).

Run the OntoGPT using Gemma 4 model

```
$ ollama list
```

NAME	ID	SIZE	MODIFIED
gemma4:26b	5571076f3d70	17 GB	4 days ago

```
$ ontogpt extract \  
-i allrecipes-spinach-feta-turkey-burgers.txt \  
-t recipe.Recipe \  
-m ollama/gemma4:26b \  
-O yaml \  
-o spinach-feta-turkey-burgers.yaml
```

PARAMETERS

-m Use the local model instead of cloud-based GPT.

Produce Grocery Item using OntoGPT

```
$ ontogpt extract \  
-i grocery-fudge-mint-cookies.txt \  
-t grocery-item.yaml \  
-m ollama/gemma4:26b \  
-O owl \  
-o grocery-fudge-mint-cookies.owl
```

PARAMETERS

-t The path to your own schema file (required).

Take Home Messages

- SPIRES drafts the whole structured knowledge base from the text in one run. You no longer build it term-by-term.
- The three expertise skills are split apart:
 - the LinkML schema captures the modelling expertise
 - the LLM supplies broad domain reading and modelling skills
 - the grounding handles the vocabulary recall
- The scarce expert moves from author to reviewer; and many documents can run in parallel.
- Grounding is done automatically by OAKlib (FOODON, UO, UBERON, MONDO, etc.) and every term is mapped to a real ontology ID, not invented by the LLM.

Reading Materials

- **Paper:** “Structured Prompt Interrogation and Recursive Extraction of Semantics (SPIRES): a method for populating knowledge bases using zero-shot learning” DOI: <http://dx.doi.org/10.1093/bioinformatics/btae104>.
- **Call Recording:** “OntoGPT: A framework for working with ontologies and large language models”. Accessible via <https://www.youtube.com/watch?v=rt3wobA9hEs&t=1955s>
- **Call Recording:** “Staying grounded: assembling structured biological knowledge with help from large language models”. Accessible via <https://www.youtube.com/watch?v=z38II6WyBsY>
- OntoGPT documentation: <https://monarch-initiative.github.io/ontogpt>
- Source Code: <https://github.com/monarch-initiative/ontogpt>